

A Guide to AI Inference Engineering



BYTEBYTEGO

JUN 15, 2026



254



3



10

Sh

[FeatureOps Summit 2026 - Feature management in the AI Era \(Sponsored\)](#)



https://blog.bytebytego.com/p/a-guide-to-ai-inference-engineering?utm_source=post-email-title&publication_id=817132&post_id=198890332&utm_campaign=ema... 2/19

Key Themes:

AI Safety Nets: Guardrails for the flood of automated code.

Edge Resilience: Sub-millisecond evaluation at scale.

Continuous Flow: Moving past the “fixed-release” mindset. Register today to master the tools and patterns required for a fail-safe release environment.

Every time an LLM generates a response, two operations run in sequence on the same GPU. The first processes the input prompt and emits a single token. The second produces every token after that, one at a time.

From the outside, they look like stages of one process. However, inside the hardware they have opposite bottlenecks. One is limited by raw compute. The other is limited by how fast data moves through memory. Most of the engineering work that makes production AI systems fast exists because of this split, and the techniques used to handle it are what inference engineering is built around.

Inference engineering is the discipline of running trained AI models in production efficiently. The work spans low-level GPU code, model serving frameworks, and cloud infrastructure that ties them together. Engineers in this field optimize for some combination of latency, throughput, cost, and quality, with the specific mix depending on the product they support. A few years ago, this work happened almost entirely inside frontier AI labs. Today, it has become a broad specialty that any company running serious AI workloads invests in.

In this article, we will walk through how inference works and why the field's optimization techniques exist.

Disclaimer: This post is based on publicly shared details from various sources. Please comment if you notice any inaccuracies.

The Rise of Inference Engineering

Three years ago, inference engineering was a specialty practiced almost entirely in frontier AI labs. The work concerned a small group of engineers building closed models that the rest of the industry consumed through APIs. That picture has shifted dramatically since 2024.

Open models drove the change. Hugging Face, the public registry for AI models, now hosts well over two million open models, roughly 25 times what existed five years ago. Open releases like DeepSeek V3 have closed the capability gap with closed models, giving companies a real choice between paying for a closed API and running an open model themselves.

Self-hosting open models brings three operational advantages over closed APIs:

- Latency profiles can be tuned for the workload pattern of a specific product where public APIs optimize for general throughput across many customers.
- Uptime can reach four nines or better with dedicated deployments, compared favorably to the two nines typical of public APIs.
- Costs typically drop by around 80 percent at scale once volume justifies engineering investment.

The result is that companies across many categories now build serious inference stacks, including AI-native startups, established products integrating AI into existing workflows, and even traditionally cautious sectors like healthcare.

Cursor offers a representative example. The team built Composer 2.0 on top of an open model, applying extensive inference engineering to deliver autocomplete latency below what closed APIs offer.

The Two Phases of LLM inference

Understanding why inference engineering looks the way it does starts with understanding what actually happens when a prompt arrives at an LLM. The process splits into two phases with very different physical demands on the GPU.

A token is the atomic unit that an LLM works with. Roughly, it is a word or word fragment. The word “inference” might be one token, while “engineering” might be split into two. Latency metrics that mention tokens per second are counted in this unit.

The first phase is called prefill.

The model takes the entire input prompt and runs it through every layer of weight parallel. Two outputs come out of this burst, namely the first token of the response and the KV cache, which is a structure that stores intermediate values from the attention mechanism so they can be referenced as more tokens get generated.

Prefill is compute-bound. The GPU’s math units are the limiting factor because every input token gets processed simultaneously through every layer of the model, throwing more raw computational power at this phase makes it faster. The metric that captures prefill performance is time to first token, or TTFT. That brief pause between sending a prompt to ChatGPT and seeing the first tokens appear is prefill in action.

The second phase is the decode phase. The model generates each subsequent token one at a time, running a full forward pass through every layer of weights for every token. Each new token depends on every token before it, which makes the process fundamentally sequential, and the GPU does this thousands of times for a full response.

Decode is memory-bandwidth-bound. Math throughput sits mostly idle while the GPU spends its cycles reading model weights from memory for each forward pass, with the bottleneck living in data movement rather than arithmetic. The metric that captures decode performance is tokens per second, or TPS. The streaming pace of a long response is the decode phase at work.

Two Phases of LLM Inference

Prefill

Compute-bound once per request

What is the weather today ?

Input prompt (all tokens processed in parallel)

*Forward pass
through every layer*

First token

KV cache

Metric: Time to First Token(TTFT)

Decode

KV cache (read)

*Generate
one token at a time*

Sunny

today

with

mild

winds ...

loop
per
token

Since prefill and decode have opposite bottlenecks, a technique that accelerates one phase often has minimal impact on the other. This is why benchmarks report TTFT and TPS as separate numbers, with performance on each phase measured independently.

This split is also the structural insight that organizes the rest of inference engineering. Once prefill and decode are understood as two distinct operations, the field of optimization techniques sort themselves into three groups: those that accelerate prefill, those that accelerate decode, and those that rebalance the two against each other.

The picture above is somewhat simplified. Real inference engines run batch scheduling, and other complexity layered on top, and the prefill-decode split happens underneath all of it, which is why it serves as the foundation for the rest of this article.

Optimization Techniques

With the prefill-decode split in mind, the major techniques in inference engineering become much easier to organize. Each one accelerates a specific phase, attacks latency for different reasons, or restructures the system around the split itself.

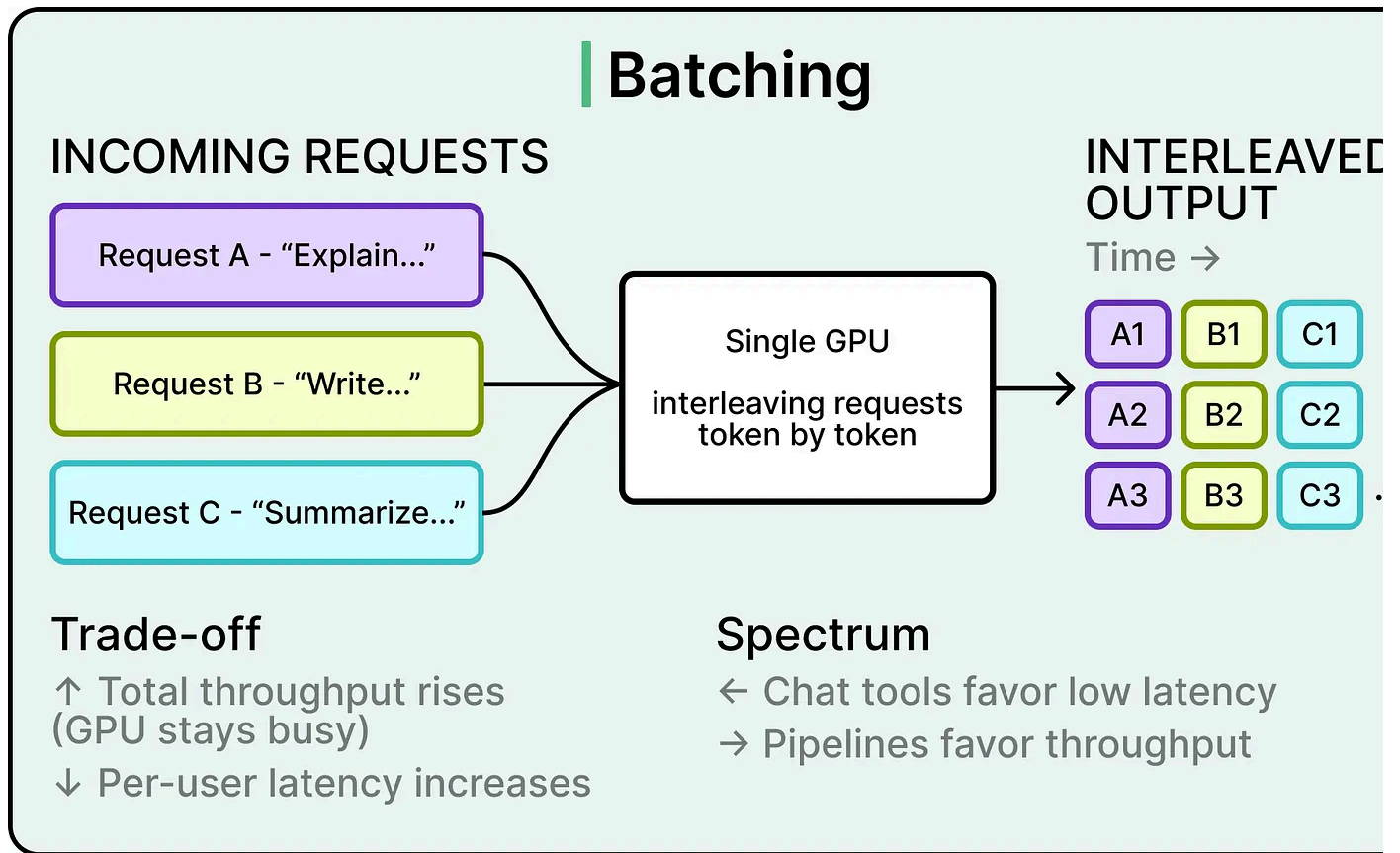
Let us cover each of the six techniques in detail.

Batching

Batching is the most basic way to scale a single GPU's output. The inference engine weaves multiple requests together, token by token, so one GPU can serve many users at once. Throughput rises significantly because the GPU's compute capacity gets fully utilized instead of sitting idle between requests.

The cost is paid in per-user latency.

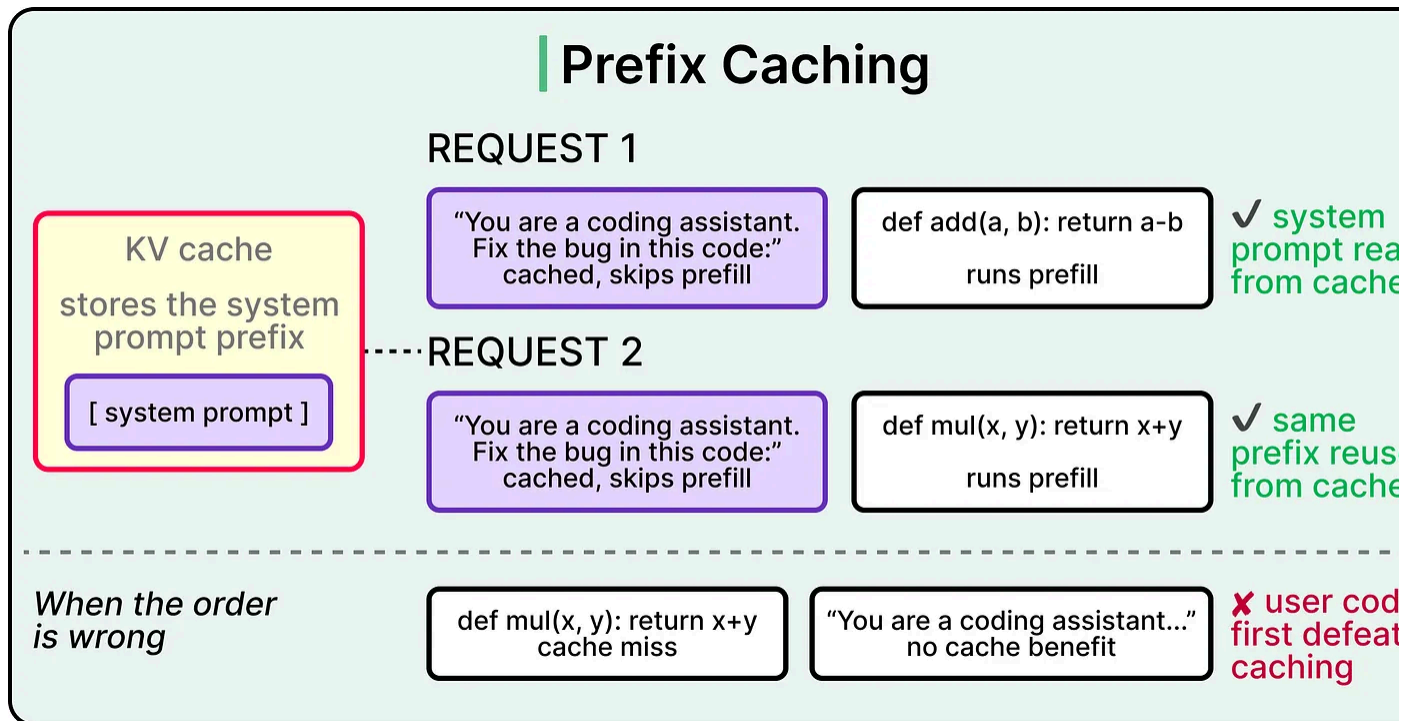
A single user on an unbatched system gets the lowest possible response time, while the same user on a heavily batched system waits longer because the GPU is serving other requests. This trade-off is the primary tension that every other technique navigates around, and different products land at different points on the spectrum, with consumer chat tools favoring lower latency and batch processing pipelines favoring higher throughput.



Prefix Caching

Prefix caching accelerates prefill by reusing KV cache values across requests. When two prompts share an opening segment, like a long system prompt that is identical across thousands of requests, the engine computes that prefix once and reads from cache thereafter. This is why API providers charge less for cached input tokens.

The catch is that the cache helps from the start of the sequence up to the first non-matching token. If the very first token differs between two prompts, prefix caching delivers zero savings even when the rest of the sequence is identical. Therefore, prompt structure has direct cost and latency implications, and putting variable input late in the prompt while keeping shared content early gives the cache something to work with.



Quantization

Quantization helps both phases of inference, though for different reasons.

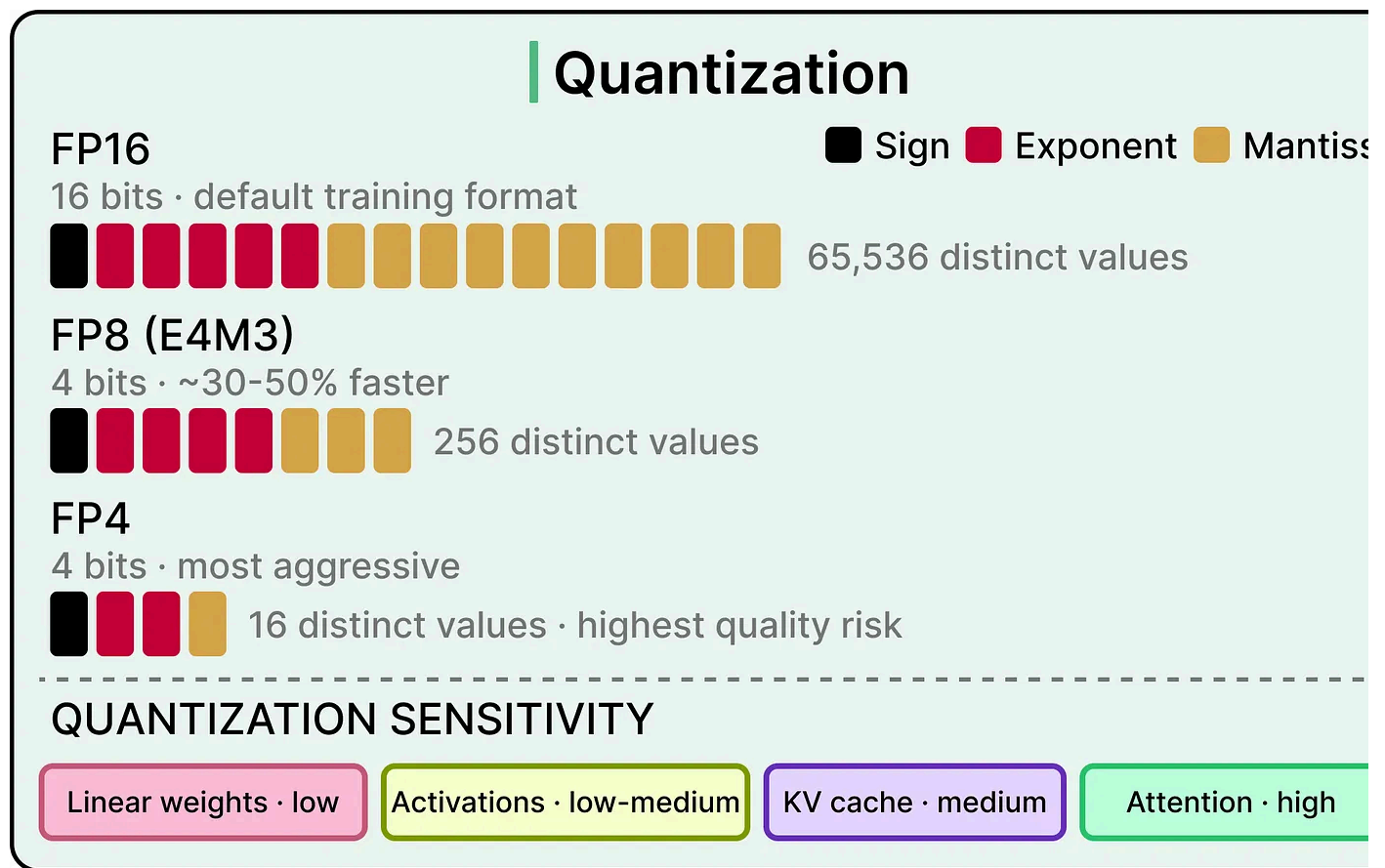
The basic move is storing model weights in a lower-precision number format. Most modern models train in 16-bit floating-point, and quantization compresses the values down to 8-bit or 4-bit representations, which means smaller weights occupy less memory and requiring less data movement.

Prefill speeds up because lower-precision math operations run faster on specialized math units inside modern GPUs. Decode speeds up because reduced memory bandwidth pressure means weights are loaded from memory more quickly forward pass. A typical step down in precision yields roughly 30 to 50 percent better performance, with the exact gain depending on the model and the technique applied.

The cost is potential quality degradation, and different parts of a model tolerate quantization differently.

Linear weights handle it well, activations are somewhat more sensitive, the KV cache is more sensitive still, and attention layers are the most sensitive of all. The reason that small precision errors in attention layers compound across the sequence tokens, with each token's calculation building on the previous ones, so even small errors snowball into meaningful quality loss over a long response.

Most production setups leave attention at full precision for this reason. The bulk of the engineering work in quantization comes down to figuring out which parts to compress and how aggressively.



Speculative Decoding

Speculative decoding accelerates the decode process by exploiting an asymmetry. Generating a token from scratch is expensive, while verifying whether a candidate token matches what the main model would produce is much cheaper. The Sud

analogy works here, where solving the puzzle takes effort, while checking a finished puzzle is fast.

In speculative decoding, a smaller draft model predicts the next several tokens, and the main model verifies all of them in a single forward pass, accepting the ones that match its own predictions and rejecting the rest. The result is multiple tokens emerging from a single forward pass through the main model, where one would normally appear.

Speculative decoding improves TPS while leaving TTFT unchanged, because the pipeline still runs normally. The technique also works best at smaller batch sizes, when the GPU has spare compute capacity to spend on verification. At larger batch sizes, with many requests being served at once, the GPU is already saturated, and speculative decoding gets dynamically disabled because every cycle is needed for the main workload.

Speculative Decoding

STEP 1-DRAFT TOKENS

Smaller draft model predicts the next several tokens



STEP 2-VERIFICATION

Main model verifies all draft tokens in a single forward pass



STEP 3-FINAL OUTPUT

Accepted draft tokens plus one newly generated correction



3 tokens produced in 1 forward pass instead of 1

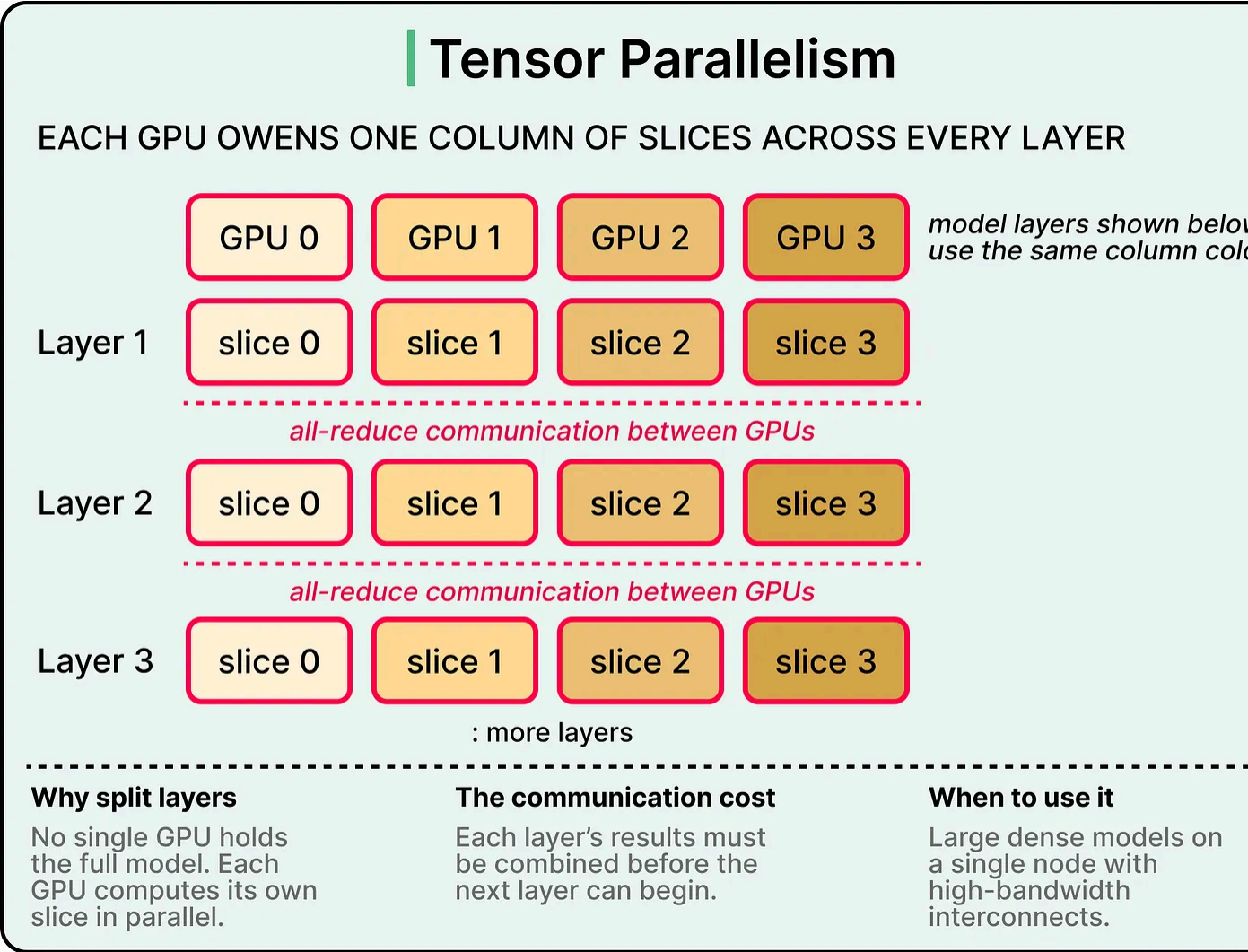
Parallelism

Parallelism techniques let large models run across multiple GPUs when a single GPU falls short, either because the model is too big to fit in memory or because single-GPU latency is too high. Two main approaches dominate the open model landscape: tensor parallelism and expert parallelism.

Tensor parallelism splits each layer of the model across multiple GPUs. Every GPU holds a fragment of every layer, and the GPUs share the work for each forward pass. This requires high-bandwidth interconnects between the GPUs, like NVIDIA NVLink, because results need to be combined after every layer. Tensor parallelism

the default choice for serving very large dense models, where the bandwidth-hur communication is offset by the speedup from sharing per-layer work.

Expert parallelism applies specifically to mixture-of-experts models, where on subset of the model’s parameters activate for each token. Different experts distributed across different GPUs, and tokens get routed to whichever experts t need. The communication overhead is lower than tensor parallelism because exp operate independently, which makes expert parallelism well-suited for multi-n deployments where bandwidth is more limited. Most production deployme combine both, using tensor parallelism within a node and expert parallelism ac nodes.



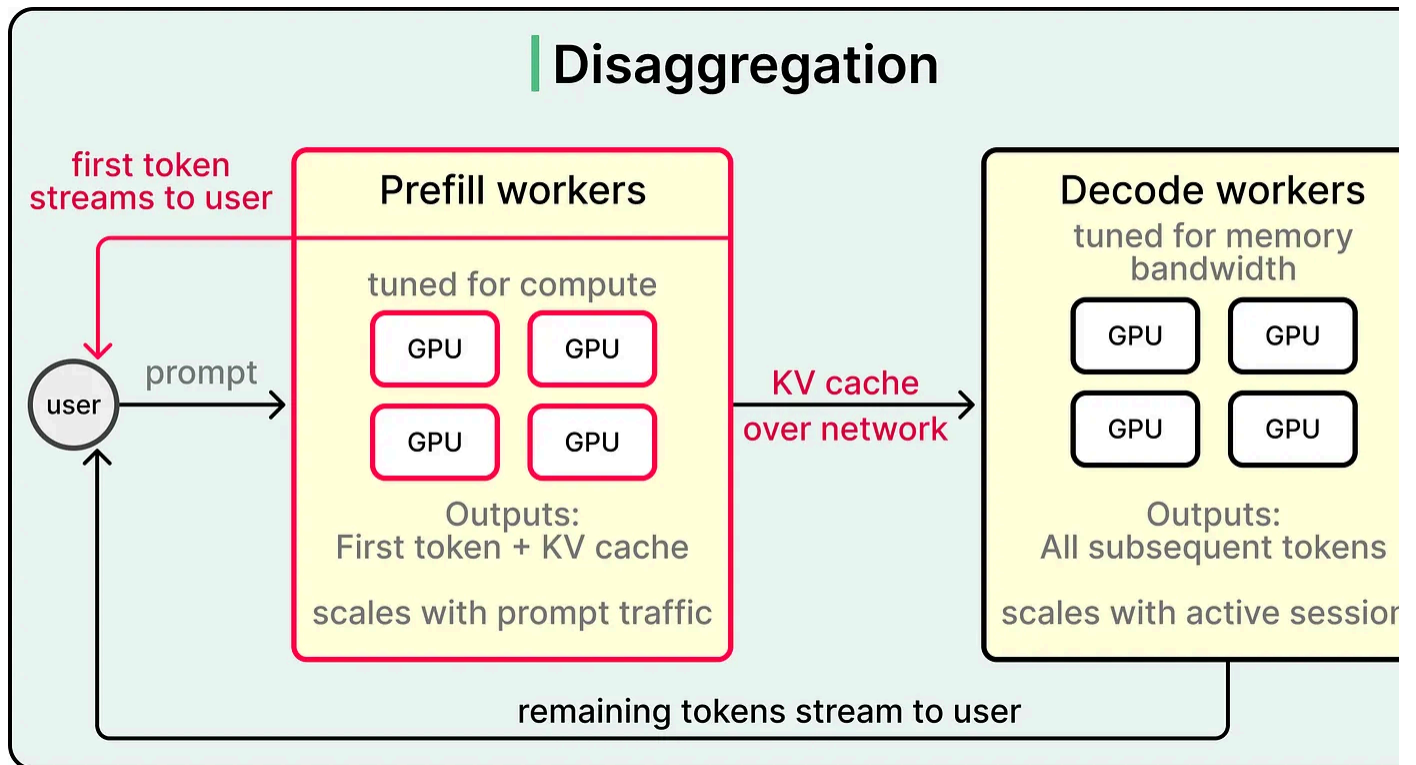
Disaggregation

Disaggregation takes the prefill-decode split literally. The idea is to run prefill on set of GPUs and decode on another, with the KV cache shipped between them over network. Each set uses hardware tuned to its specific bottleneck, and each set scales independently based on its own traffic pattern.

The flow becomes a three-step process:

- The prefill engine takes the input sequence and produces both the first token and the KV cache.
- The cache gets sent over a fast interconnect to the decode engine, and the decode engine handles every subsequent token.
- In conditional disaggregation, short or already-cached requests skip the prefill entirely and run on the decode engine alone, which performs better against real world traffic that includes a mix of long and short prompts.

Disaggregation is the most architectural of the techniques covered here. It treats prefill and decode as separate services with separate operational concerns, giving operators independent levers to scale each one. Companies running large-scale inference often consider this a near-mandatory step once their workload mix is well understood.



When to Invest in Inference Engineering

Putting these techniques into production is a serious task, and combining them adds further complexity. The question every engineering team has to answer is whether to take on this work or whether off-the-shelf APIs are still the right choice. The answer depends on the stage of the product.

Early in building an AI product, off-the-shelf APIs from established providers are almost always the right choice. Meaningful optimization requires real constraint work against, and early-stage products tend to have fuzzy assumptions about traffic patterns, latency requirements, and unit economics. Engineering effort at this stage is better spent shipping product, since the complexity of running a custom inference stack slows down iteration when iteration speed is what actually matters.

Three signals usually indicate the equation has shifted:

- API costs have grown into a meaningful expense line.
- Latency requirements have moved past what closed APIs can deliver.

- Reliability needs have started to exceed what vendor SLAs offer.

Cursor handled this transition well. Sub-second autocomplete latency was the product itself, and closed APIs aim for general throughput across many customers, while the code completion model demands a specific shape of speed. Self-hosting an open model and applying inference engineering across the stack made the latency target reachable and the investment paid back because the constraints were real and the workload was well understood.

Conclusion

LLM inference is two operations with opposite physical constraints.

Prefill is compute-bound and runs once per request. Decode is memory-bandwidth bound and runs once per token. Most of the techniques in inference engineering exist because of this split, and grasping it makes the rest of the field much easier to navigate.

Each technique covered above fits into the prefill-decode framework:

- Batching trades per-user latency for total throughput.
- Prefix caching cuts prefill work when prompts share opening segments.
- Quantization compresses model weights to help both phases.
- Speculative decoding squeezes more tokens out of decode by exploiting compute.
- Parallelism scales models across multiple GPUs.
- Disaggregation runs prefill and decode on separate hardware altogether.

Layered on top of all this is the build-versus-buy question. Off-the-shelf APIs remain the right choice for most products in their early stages, while self-hosting starts

make sense when API costs grow into a real expense line, when latency requirements outgrow what closed APIs can deliver, or when reliability needs exceed vendor SLA

References:

- [Hugging Face Hub Documentation](#)
- [DeepSeek-V3 Technical Report](#)
- [Cursor — Composer: Building a fast frontier model with RL](#)
- [Cursor — Introducing Composer 2](#)
- [DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving](#)
- [Efficient Memory Management for Large Language Model Serving with PagedAttention](#)
- [Anthropic — Prompt Caching Documentation](#)
- [NVIDIA TensorRT Documentation](#)
- [Fast Inference from Transformers via Speculative Decoding](#)
- [Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism](#)
- [NVIDIA Megatron-Core Developer Guide — Tensor Parallel](#)
- [Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer](#)
- [NVIDIA NVLink and NVLink Switch](#)
- [Anthropic Claude API Documentation](#)

Subscribe to ByteByteGo Newsletter

Explain complex systems with simple terms, from the authors of the best-selling system design book series. Join over 1,000,000 friendly readers.

[Subscribe](#)

By subscribing, you agree Substack's [Terms of Use](#), and acknowledge its [Information Collection Notice](#) and [Privacy Policy](#).



254 Likes • 10 Restacks

Discussion about this post

[Comments](#)[Restacks](#)

Write a comment...



Anny He 2d

Hmm you can have both, eg. use open models (eg. llama, kimi) via API in a hosted remote environment like AWS Bedrock. This way you delay building out the infrastructure while using and avoid paying frontier model fees.

♡ LIKE 💬 REPLY



Mitchell Kosowski 2d

Best explanation of the prefill/decode split I've read: framing decode as memory-bandwidth-l makes speculative decoding and quantization click instantly. Saving this as a reference. Than writing it.

♡ LIKE 💬 REPLY



1 more comment...

© 2026 ByteByteGo · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture